

LBFGS Reduction Crashes in MTS2D

Reproducibility, correction-count sensitivity, and recovery design

SimulationScripts investigation

21 July 2026

Abstract

Several reversibility simulations stopped during an L-BFGS minimization after a reduction step produced an explosively large state. The failure is deterministic when the same checkpoint and the same number of correction vectors are used: replaying with $M=7$ reproduces the same crash signature. Changing only the correction count avoids the failure over the tested load interval; $M=6$, as well as $M=3, 5, 8$, and 10 , completed the test interval from the same checkpoint. This report records the evidence, its limitations, and a proposed one-time fallback that tries the paired correction count ($7 \leftrightarrow 6$) before giving up. The fallback is described here only; it has not yet been added to the simulation.

1 Problem and scope

The investigation concerns the `reversibilityProtocolTest` family with a 100×100 mesh, periodic boundary conditions, `edgeFlip` reconnection, and L-BFGS minimization. The important failing configurations used a load increment of either 10^{-5} or 10^{-6} . A reduction guard reported an exploded state during minimization and the process terminated with exit code `-6`.

The clearest original failure was seed 2 of the 10^{-5} configuration. Its nearest usable normal checkpoint was

```
/Volumes/data/MTS2D_output/reversibilityProtocolTest,s100x10010.14,1e-05,1.  
OPBCedgeFlip2LBFGSEpsx1e-07energyDropThreshold1e-05s2/dumps/dump_10.61.xml.gz
```

The crash occurred at load 0.61852 (serialized checkpoint load 0.61851), at load step 47853. The reduction diagnostic identified $eIndex = 6864$ and $m3Nr = 520$. The failed minimization reached an energy of approximately 2.6×10^{22} and a maximum force of approximately 1.2×10^{17} , far outside the normal scale of the run.

Before entering the debug replay, MTS2D was extended to save a nonzero checkpoint named `crash_<normal-dump-name>` in the simulation's dumps directory. This preserves the state at which the reduction failed and makes an independent replay possible. That crash-checkpoint feature is implemented and was used in this investigation.

2 Experimental procedure

Each test started from the same valid normal dump. The maximum load was capped five increments past the suspected failure so that the tests did not need to repeat the entire simulation. The procedure was:

1. run with the original $M=7$ configuration until the reduction failure, producing a crash dump;
2. restart from that crash dump with $M=7$ and check whether the same failure is obtained;
3. restart from the identical crash dump with a changed correction count and check whether the run reaches the cap.

The automated version of this procedure is in `debug_crashed_simulations.py`. It writes each phase to a separate output tree, so the original data and the checkpoint used by the comparison are not overwritten.

3 Evidence for deterministic reproducibility

For the seed-2 checkpoint described above, both the first $M=7$ run and the independent replay reported the same final reduction signature:

Quantity	Value
Element index	6864
$m3$ counter	520
Crash load	0.6185199999998207
Load step	47853
Process result	exit code -6

The same signature was also reproduced for the 10^{-5} seed-0 case: $eIndex = 15802$, $m3Nr = 313$, load 0.17444000000003446, and load step 3445. Thus the reproducibility is not an artefact of a single seed. The matching signatures, together with identical restart data, show that the crash is deterministic for a fixed checkpoint and fixed M .

4 Changing the correction count

Starting from the seed-2 crash checkpoint, the following direct tests were made:

Test	Result	Interpretation
$M=7$	Reproduced the exact crash	Baseline is deterministic
$M=3$	Reached the capped load 0.61857	No crash in the tested interval
$M=5$	Reached the capped load 0.61857	No crash in the tested interval
$M=6$	Reached the capped load 0.61857	No crash in the tested interval
$M=8$	Reached the capped load 0.61857	No crash in the tested interval
$M=10$	Reached the capped load 0.61857	No crash in the tested interval
Looser minimization tolerances	Reached the cap	Also changes the numerical path
Doubled load increment	Reached the cap	Useful sensitivity check, but not identical dynamics
Conjugate gradient cross-check	Passed the target step	Supports an L-BFGS-specific sensitivity, but is not a proof
FIRE cross-check	Stopped after more than 235,000 function calls	Inconclusive with the tested settings

The most controlled comparison is therefore $M=7$ versus $M=6$ from the same crash dump. The correction count is the only changed minimizer setting in that comparison, and $M=6$ avoided the observed failure. The additional values ($M=3, 5, 8$, and 10) show that the result is not unique to one specially chosen alternative.

An automated four-case follow-up was started for all existing crash-like outputs. At the time of writing, the two 10^{-5} cases had both reproduced their crashes with $M=7$ and completed with $M=6$. The 10^{-6} seed-1 and seed-2 cases were still in their initial $M=7$ replay, approaching their respective crash loads. Their unfinished status is not used as evidence in the conclusions above.

5 Interpretation and limitations

Changing M changes the amount of quasi-Newton history retained by L-BFGS and therefore changes the sequence of trial steps. A flat or poorly conditioned local energy landscape can make those paths diverge even when they start from exactly the same mesh. The observations are consistent with a numerical-stability or path-sensitivity issue in the minimization, rather than evidence of a random crash.

The result does not, by itself, prove that the simulation state is bug-free. A simulation bug could be exposed only by particular minimizer trajectories, and the alternative- M runs were tested only for a short interval after the crash. For that reason, the crash dump, exact $M=7$ replay, and comparison with an independent minimizer remain important diagnostics.

6 Proposed recovery behavior

The following small recovery feature is proposed for a later implementation; it has not been added in this report’s work:

1. On a reduction/minimization crash, first save the crash checkpoint with a `crash_` prefix followed by the normal dump name, and retain it permanently for diagnosis.
2. Reset to that checkpoint and retry once with the paired correction count: use $M=6$ when the failed run used $M=7$, and use $M=7$ when the failed run used $M=6$.
3. If the retry completes the minimization, resume the ordinary simulation from the recovered state and continue writing normal dumps.
4. If the retry also fails, stop with the original error and both diagnostic states available.

The retry should be explicitly recorded in the log, including the original and alternate M values, the crash-dump path, and the outcome. The crash dump must remain untouched even when the alternate run succeeds. Only one deterministic fallback is recommended; randomly changing M repeatedly would make failures harder to reproduce and would obscure the diagnostic record.

7 Conclusion

The key result is a controlled, repeatable comparison: the same crash checkpoint with $M=7$ produces the same reduction failure, while changing only the correction count to $M=6$ avoids it over the tested interval. This makes a one-time paired- M fallback a reasonable resilience feature, provided that the original crash checkpoint and failure are preserved. The feature is intentionally deferred until the current evidence-gathering campaign is complete.